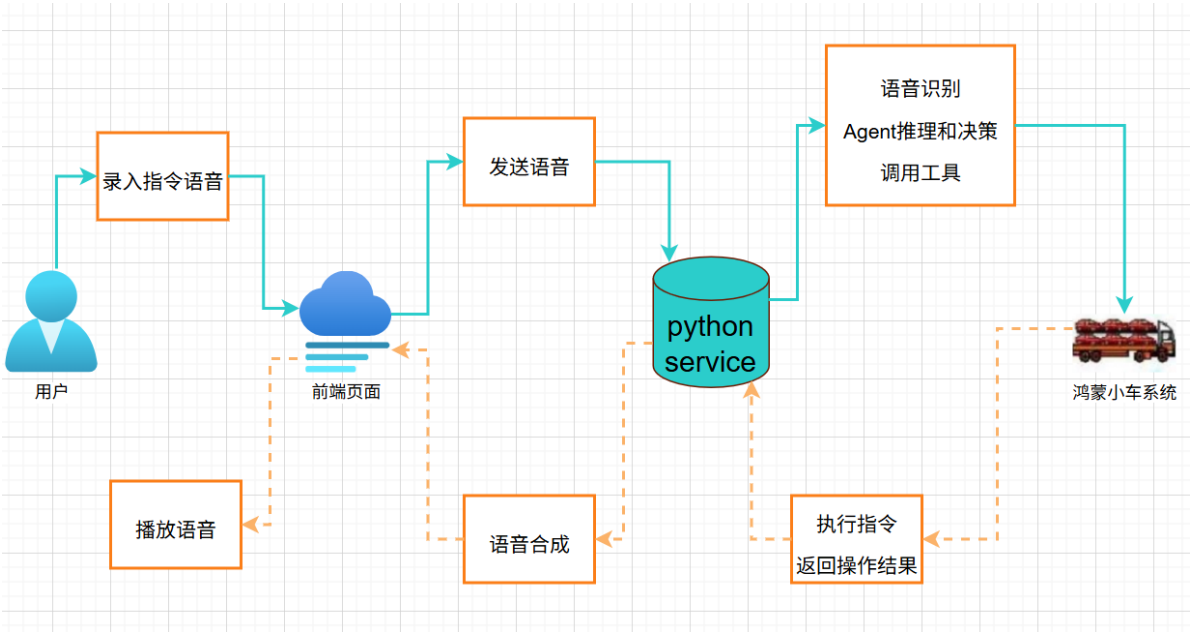
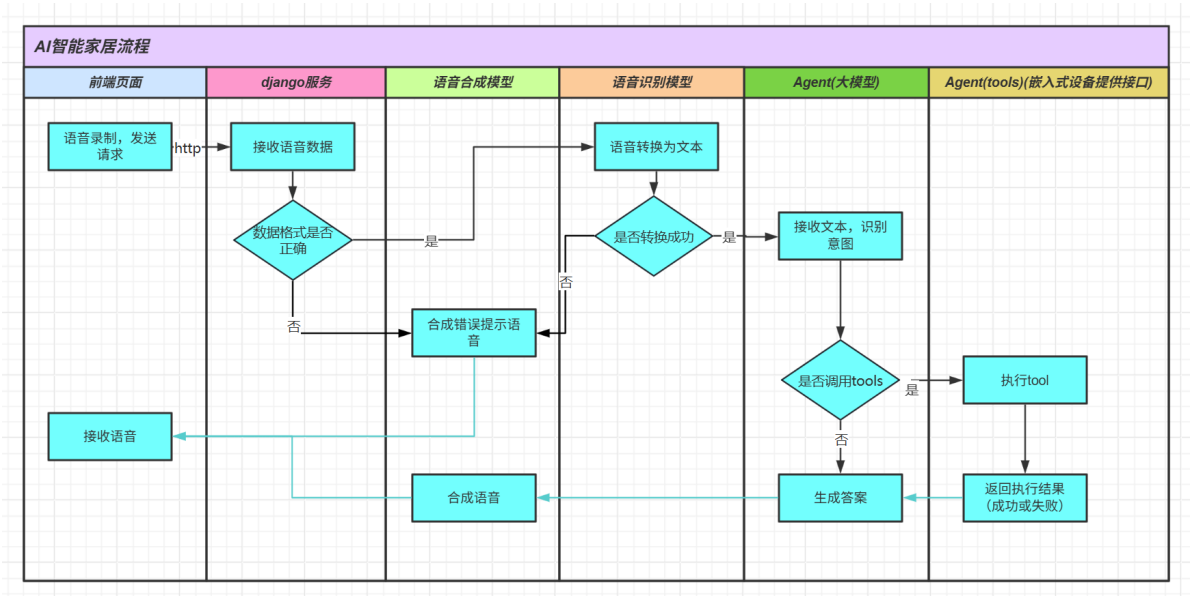
# Agent-鸿蒙小车

## 1.整体介绍

### 1.1 系统架构




### 1.2 系统流程




### 1.3 AI技术栈

**环境：**

Anaconda：2024

**IDE：**

Pycharm：2024

HBuilderX：latest

**技术栈：**

后台服务：

Python：3.11

Django：latest

LangChain/LangGraph：1.2.0

前端页面：

html、css、javascript

## 1.4 安装

Anaconda：详见安装包说明

Pycharm：详见安装包说明

## 1.5 虚拟环境

创建虚拟环境：

打开anaconda prompt命令提示符对话框，输入：

```
conda create -n dly_env python=3.11
```

说明：dly_env是虚拟环境名称

激活虚拟环境：

输入：

```
conda activate dly_env
```

## 1.6 安装基础依赖包

需要安装的基础依赖包如下：

```
pip install django-cors-headers
pip install requests
pip install "langchain[openai]"
```

# 2.智能体核心概念：它是什么？

Agent 的核心思想是使用一个大语言模型（LLM）作为其“大脑”或“决策核心”，来理解用户需求、制定计划、选择工具（Tools）、执行动作（Actions）、并观察结果，最终完成一个复杂的任务。

简单来说，一个 Agent 的运作模式是：
**思考（Think） -> 行动（Act） -> 观察（Observe） -> ... -> 最终完成**

这非常类似于人类的思考方式。比如，当被问到“奥利奥卡路里高吗？”时，一个 Agent 可能会这样工作：

1. **思考**：“用户问的是奥利奥的卡路里。我自己没有实时数据，但我可以调用网络搜索工具。”
2. **行动**：调用 `google_search` 工具，输入查询关键词 “Oreo calorie content”。
3. **观察**：工具返回了搜索结果，例如 “一份（3块）奥利奥含有约 160 卡路里”。
4. **再思考**：“我得到了数据。现在需要判断这个数值是否算‘高’。我可以调用一个计算工具，将其与常见零食对比，或者直接用我的知识进行推理。”
5. **再行动/回答**：基于推理，生成最终答案：“一份（3块）奥利奥大约含有160卡路里。相比于一些水果（如一个苹果约95卡），这个热量相对较高，属于能量密度较高的零食，应适量食用。”

## 2.1 为什么需要 Agents？解决了什么问题？

尽管LLM本身非常强大，但它有固有的局限性：

1. 知识滞后与幻觉：如前所述，LLM的知识有截止日期，且会编造信息。
2. 无法执行外部动作：LLM可以写代码，但不能真正运行代码；它可以描述如何发送邮件，但不能真的帮你发邮件。
3. 数学计算、实时信息获取等能力弱：LLM可能不擅长精确计算或获取最新信息。

Agents 通过“工具使用（Tool Use）”能力，将 LLM 的推理和语言能力与外部工具的强大功能结合起来，突破了LLM自身的限制，使其成为一个真正的“数字助手”。

# 3.核心组成部分

一个典型的 Agent 由以下几个关键部分构成：

1. **LLM (大脑)**

   负责所有的推理和决策。它决定下一步该做什么，使用哪个工具，以及如何解释工具返回的结果。主要功能包括：

   - 理解用户意图和任务需求
   - 进行逻辑推理和规划
   - 决定是否需要调用工具
   - 生成工具调用的正确参数
2. **工具 (Tools)**

   Agent 可以调用的外部函数。这是 Agent 能力的扩展。包括：

   - 搜索工具、计算器、API调用等预置工具
   - 自定义工具：可封装任意业务逻辑（如数据库操作、文件处理）
   - 关键特性：每个工具需明确定义功能描述和参数规范，便于LLM准确调用
3. **记忆（Memory）**：

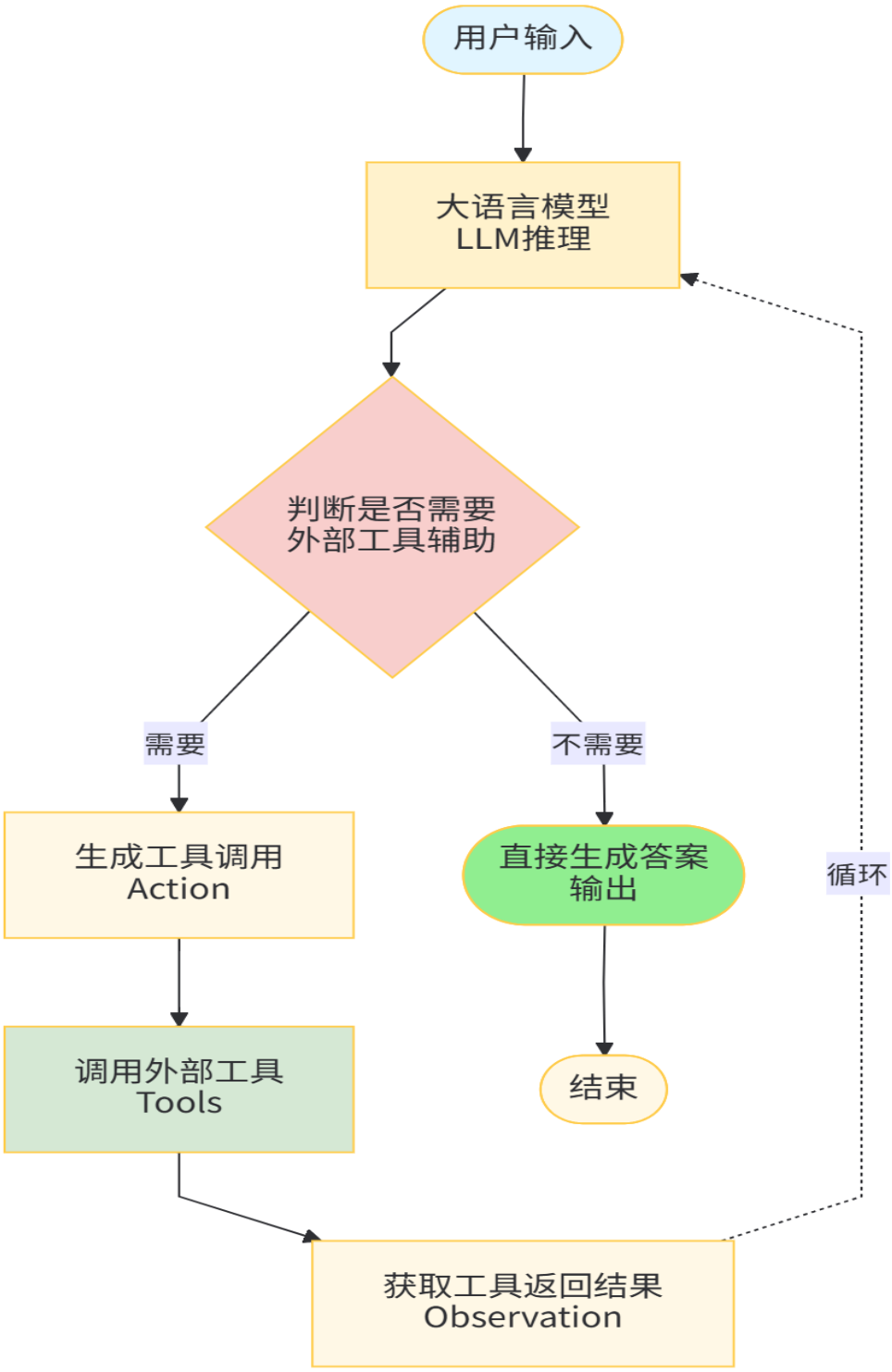
   - Memory为Agent提供上下文感知能力，使其能够记住之前的交互历史并基于上下文做出决策。
   - LangChain中的记忆系统分为短期记忆（维护当前对话的上下文） 和长期记忆（支持跨对话会话的知识持久化）。
   - 在LangChain1.0中，记忆系统与LangGraph的状态管理深度融合，支持更复杂和结构化的状态保持
4. **代理执行器 (Agent Executor)**

   - 在LangChain 1.0的架构中，AgentExecutor继续扮演执行协调器的关键角色，负责选代运行代理直至满足停止条件。
   - 新版本中，AgentExecutor获得了更强大的流程控制能力，包括循环控制（通过max_iterations参数防止无限循环）、异常处理（可配置的解析错误处理）和可观测性。

# 4.工作原理

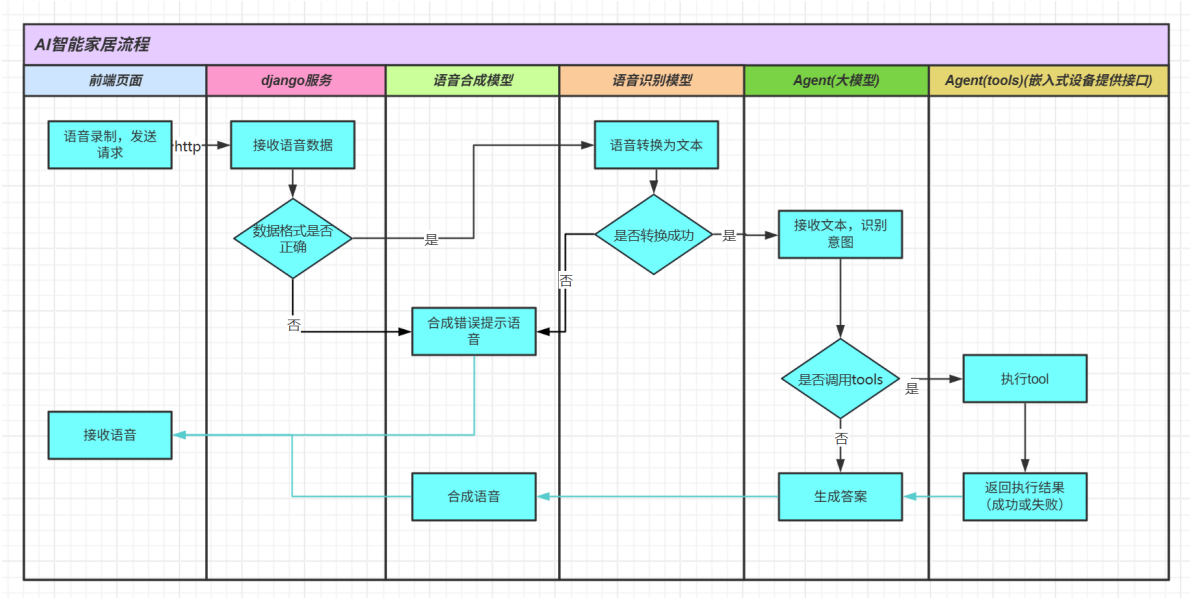
**Agent 工作原理：**

- **输入解析**：Agent首先接收用户输入，解析其意图和关键参数。输入解析器负责标准化用户请求，提取可供工具使用的结构化参数。
- **LLM推理**：解析后的输入与当前状态（包括记忆和历史记录）一起传递给LLM进行推理。LLM基于预设的提示词模板分析情况，决定下一步行动——直接回答还是调用工具。
- **工具调用**：如果LLM决定调用工具，AgentExecutor会执行相应的工具函数，并获取执行结果。工具调用可能涉及外部API访问、数据库查询或计算操作等。
- **观察与迭代**：工具返回的结果作为"观察"被反馈给Agent，这些信息与之前的状态一起形成新的上下文，传递给LLM进行下一轮推理。这个循环持续进行，直到LLM认为已获得足够信息来生成最终答案。
- **结果整合**：当循环结束时，Agent整合所有获取的信息，生成格式化的最终答案返回给用户。同时，重要的交互信息可能被存储在记忆系统中，供未来会话使用。



# 5.功能实现

## 5.1 流程




## 5.2 阿里云百炼API Key

创建阿里云百炼API Key网址：[https://bailian.console.aliyun.com/?tab=globalset#/efm/api_key](https://bailian.console.aliyun.com/?tab=globalset#/efm/api_key)

帮助文档：[https://help.aliyun.com/zh/model-studio/get-api-key?spm=a2c4g.11186623.help-menu-2400256.d_2_0_0.230611fc9mwk6f&scm=20140722.H_2712195._.OR_help-T_cn~zh-V_1](https://help.aliyun.com/zh/model-studio/get-api-key?spm=a2c4g.11186623.help-menu-2400256.d_2_0_0.230611fc9mwk6f&scm=20140722.H_2712195._.OR_help-T_cn~zh-V_1)

## 5.3 env文件

```
QWEN_API_KEY=你的api key
QWEN_MODEL_NAME=qwen3-max
QWEN_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
DASHSCOPE_BASE_HTTP_API_URL=https://dashscope.aliyuncs.com/api/v1
QWEN_ASR_MODEL_NAME=qwen3-asr-flash
QWEN_TTS_MODEL_NAME=cosyvoice-v3-flash
DEVICE_BASE_URL=http://localhost:8088
```

#### env文件封装

创建一个env_util.py文件，将env文件中的键值对进行二次封装，既可以统一管理，也能避免敏感信息泄露

```
import os

from dotenv import load_dotenv

load_dotenv()

qwen_api_key = os.getenv('QWEN_API_KEY')
qwen_model_name = os.getenv('QWEN_MODEL_NAME')
qwen_base_url = os.getenv('QWEN_BASE_URL')
dashscope_base_http_api_url = os.getenv('DASHSCOPE_BASE_HTTP_API_URL')
qwen_asr_model_name = os.getenv('QWEN_ASR_MODEL_NAME')
```

```
qwen_tts_model_name = os.getenv('QWEN_TTS_MODEL_NAME')

device_base_url = os.getenv('DEVICE_BASE_URL')
```

## 5.4 语音识别模型

网址：https://help.aliyun.com/zh/model-studio/qwen-speech-recognition

安装依赖包：

```
pip install dashscope
```

```
import os
import traceback

import dashscope
from utils import env_util

dashscope.base_http_api_url = env_util.dashscope_base_http_api_url


def speech_to_text(base64_uri):
    try:
        messages = [
            {"role": "system", "content": [{"text": "将语音转换为文字"}]},  # 配置定制化识别的 Context
            {"role": "user", "content": [{"audio": base64_uri}]}
        ]
        response = dashscope.MultiModalConversation.call(
            api_key=env_util.qwen_api_key,
            model=env_util.qwen_asr_model_name,
            messages=messages,
            result_format="message",
            asr_options={
                "enable_itn": False
            }
        )
        return response.output.choices[0].message.content[0]['text'].replace('"', '')
    except Exception as e:
        traceback.print_exc()
        return None
```

## 5.5 语音合成模型

网址：https://help.aliyun.com/zh/model-studio/text-to-speech?spm=a2c4g.11186623.help-menu-2400256.d_0_4_0.3398338fbvUVB7&scm=20140722.H_2842586._.OR_help-T_cn~zh-V_1

```
import base64
import traceback
```

```python
import dashscope
from dashscope.audio.tts_v2 import *

from utils import env_util


def text_to_speech(text):
    try:
        dashscope.api_key = env_util.qwen_api_key

        # 模型和音色配置
        model = env_util.qwen_tts_model_name
        voice = "longanhuan"

        # 实例化合成器
        synthesizer = SpeechSynthesizer(model=model, voice=voice)

        # 合成文本
        audio = synthesizer.call(text)

        # 音频转base64
        base64_code = base64.b64encode(audio).decode("utf-8")

        # 保存音频
        with open('output.mp3', 'wb') as f:
            f.write(audio)

        return base64_code
    except Exception as e:
        traceback.print_exc()
        return None
```

## 5.6 千问大模型服务

网址：https://bailian.console.aliyun.com/model-market?tab=model#/model-market

安装依赖包：

```
pip install langchain-openai
```

```python
import os

from langchain_openai import ChatOpenAI
from utils import env_util


qwen_llm = ChatOpenAI(
    model=env_util.qwen_model_name,
    base_url=env_util.qwen_base_url,
    api_key=env_util.qwen_api_key,
    temperature=0.5,
    max_tokens=2048
```

```
)
```

## 5.7 创建agent_tool

创建工具tool，在tool中调用客户端提供的API接口，这里先使用模拟测试接口进行模拟测试，后续再跟客户端进行接口对接联调

接口定义：

```
/*
    设备控制接口：

        url: http://localhost:8080/device/control
        method: POST
        params: {                //json格式
            "led": "green",      //亮绿灯   "red", "blue","off"
            "fan": false,        //关风扇
            "beep": true         //开蜂鸣器
            "car": forward       //前进  backword:后退   left:左转  right: 右转  stop: 停止
        }
        response:{
            "code": 200,
            "msg": "control success",
            "data": {
                "led": "ok",
                "fan": "ok",
                "beep": "ok"
            }
        }
    数据查询接口：
        url: http://localhost:8080/device/status
        method: GET
        response: {
            "code": 200,
            "msg": "success",
            "status": {
                "fan": false,
                "led": false,
                "tem": 4,
                "hum": 55,
                "light": 3331,
            }
        }
```

### 5.7.1 mock接口

**该接口是模拟鸿蒙小车的设备状态查询和设备控制，在AI开发中临时使用，对接鸿蒙小车后该mock接口则废弃，不再使用。**

新创建一个django项目作为模拟测试的项目，在该项目中创建接口定义中的API接口

```
import json

from django.http import JsonResponse


def device_control(request):
    body_json = json.loads(request.body)

    led = body_json['led']
    fan = body_json['fan']

    beep = body_json['beep']
```

```
    car = body_json['car']

    print("led:", led, "fan:", fan, "beep:", beep, "car:", car)

    dic = {
        "code": 200,
        "data": {
            "fan": "ok",
            "led": "ok",
            "beep": "ok",
            "car": "ok"
        },
        "msg": "control success"
    }

    return JsonResponse(dic)


def device_status(request):
    dic = {
        "code": 200,
        "msg": "success",
        "status": {
            "fan": False,
            "led": False,
            "tem": 4,
            "hum": 55,
            "light": 3331
        }
    }

    return JsonResponse(dic)
```

接口路由:

在django项目的urls.py中添加:

```
# 导入接口模块
from api import device

# 添加接口路由
path('device/control', device.device_control),
path('device/status', device.device_status)
```

### 5.7.2 创建tool

工具包含以下几个组件:

| 属性 | 类型 | 描述 |
| --- | --- | --- |
| 名称 | str | 在提供给 LLM 或代理的一组工具中必须是唯一的。默认为方法名。 |
| 描述 | str | 描述工具的作用。被 LLM 或代理用作上下文。 |
| args_schema | pydantic.BaseModel | 可选但推荐，如果使用回调处理程序则为必需。它可用于为预期参数提供更多信息（例如，少量示例）或验证。 |
| return_direct | boolean | 仅与代理相关。当为 True 时，在调用给定工具后，代理将停止并将结果直接返回给用户。 |

**注意：**如果工具具有精心选择的名称、描述和args_schema，模型将表现得更好。

LangChain 支持从以下几种方式创建工具

1. Tool装饰器的函数 -- 这是最常见的。
2. 通过从 BaseTool 子类化 -- 这是最灵活的方法，它提供了最大的控制程度，但代价是需要付出更多的努力和编写更多的代码。
3. 从MCP的服务端获得工具

本案例采用Tool装饰器实现：

```
import requests
from langchain_core.tools import tool
from typing import Annotated
from utils import env_util

base_url = env_util.device_base_url
@tool
def device_control(
        led: Annotated[
            str,
            "控制灯光的开和关，可选值：green/blue/red/off，green表示亮绿灯，blue表示亮蓝灯，red表示亮红灯，off表示关闭灯光"
        ],
        fan: Annotated[
            bool,
            "控制风扇的开和关，可选值：True/False，True表示开，False表示关"
        ],
        beep: Annotated[
            bool,
            "控制蜂鸣器的开和关，可选值：True/False，True表示开，False表示关"
        ],
        car: Annotated[
            str,
            """控制设备的运行方向，可选值：forward/backward/left/right/stop，
```

```
            forward表示前进，backward表示后退，left表示左转，right表示右转，
            默认值为stop
            """
        ]
):
    """控制设备开和关操作的工具"""
    try:
        # 爬虫接口的url路径
        url = f"{base_url}/device/control"

        params = {
            "led": led,
            "fan": fan,
            "beep": beep,
            "car": car
        }

        res = requests.post(
            url=url,
            json=params,
            headers={'Content-Type': 'application/json'},
            verify=False
        )
        res_json = res.json()
        if res_json["code"] == 200:
            return "设备操作成功，结束"
        else:
            return "设备操作失败"
    except Exception as e:
        return "设备操作失败"


@tool
def device_status():
    """查询设备的当前状态，返回语义化描述"""

    url = f"{base_url}/device/status"

    res = requests.get(
        url=url,
        headers={'Content-Type': 'application/json'},
        verify=False
    )
    res_json = res.json()
    if res_json["code"] == 200:
        status = res_json["status"]
        led = status["led"]
        fan = status["fan"]
        tem = status["tem"]
        hum = status["hum"]
        light = status["light"]
        result = ""
        if led:
            result += f"灯光为打开状态"
        else:
```

```
            result += "灯光为关闭状态"

        if fan:
            result += "，风扇为打开状态"
        else:
            result += "，风扇为关闭状态"

        result += f"，温度为{tem}，湿度为{hum}，亮度为{light}"
        return result
    else:
        return "设备状态查询失败"
```

## 5.9 Agent集成

```
from langchain.agents import create_agent
from langchain_core.callbacks import BaseCallbackHandler
from langchain_core.runnables import RunnableConfig
from langchain_core.prompts import ChatPromptTemplate

from models.my_llm import qwen_llm
from tools.device_tool import device_status, device_control

agent = create_agent(
    model=qwen_llm,
    tools=[device_status, device_control]
)


class DebugCallback(BaseCallbackHandler):
    def on_llm_start(self, serialized, prompts, **kwargs):
        """大模型开始时调用"""
        print(f"{prompts[0][:]}\n\n")

    def on_tool_start(self, serialized, input_str, **kwargs):
        """Tool开始时调用"""
        print(f"Tool开始: {serialized.get('name')}，输入: {input_str}")

    def on_tool_end(self, output, **kwargs):
        """Tool结束时调用"""
        print(f"Tool结束，输出: {output}")


def agent_process(text, max_tool_calls):
    """
    执行 Agent 并强制限制工具调用次数

    Args:
        text: 用户输入文本
        max_tool_calls: 最大允许的工具调用次数（默认3次）
    """
    prompt_template = ChatPromptTemplate.from_messages([
        ("system", "你是一个鸿蒙小车助手，能够根据指令查询小车状态以及控制小车。"),
        ("human", "{question}")
    ])
```

```
    config = RunnableConfig(recursion_limit=max_tool_calls * 3 + 5, callbacks=
[DebugCallback()])

    chain = prompt_template | agent

    results = chain.invoke({"question": text}, config)
    for result in results["messages"]:
        result.pretty_print()
        if result.type == 'ai' and result.content:
            return result.content


if __name__ == '__main__':
    agent_process("打开风扇", 3)
```

说明：RunnableConfig

**RunnableConfig**是一个**运行时配置对象**，用于向可运行组件（Runnable）传递执行参数、回调处理器、元数据等配置信息。它是控制链（Chain）、代理（Agent）等组件行为的核心机制。

RunnableConfig本质上是一个**字典结构**，在调用 `invoke`、`stream`、`batch` 等方法时通过 `config` 参数传递，用于：

- **注入运行时变量**（如用户ID、会话ID）
- **附加回调监控**（调试、日志、流式输出）
- **控制执行行为**（重试策略、超时设置）
- **传递元数据**（追踪标签、运行名称）

## 4.9 逻辑集成

主要把语音识别，语音合成，Agent进行逻辑处理

```
from agent import my_agent
from speech_to_text import asr
from text_to_speech import tts


def process(base64_uri):
    text = asr.speech_to_text(base64_uri)
    if not text:
        return tts.text_to_speech("出错了，语音识别失败")

    agent_content = my_agent.agent_process(text)

    base64_code = tts.text_to_speech(agent_content)
    if not base64_code:
        return None

    return base64_code
```

## 4.10 API接口

API接口：向客户端提供方法，供客户端调用

```
语音上传接口:
    url: http://localhost:8000/audio/upload
    method: POST
    params: {
      "base64_uri": "data:audio/mpeg;base64"
      "data": "XXXX"    //参数以json格式传递，base64_uri是将语音文件(mp3格式)转换后的base64编码，"data:audio/mpeg; base64,"是base64的头信息
    }
    response:{
      "code":"200",        //状态码，200-成功，500-失败
      "msg":"操作成功",    //提示信息
      "data":"data:audio/mpeg; base64,XXXX"    //返回大模型应答语音(mp3格式)的base64编码，"data:audio/mpeg; base64,"是base64的头信息
    }
```

```python
import json

from django.http import JsonResponse

from service import ai_service


def upload(request):
    base64_json = json.loads(request.body)
    base64_uri = base64_json['base64_uri']
    base64_code = base64_json['data']
    base64_uri = base64_uri + base64_code

    result = ai_service.process(base64_uri)
    if not result:
        data = {"code": 500, "msg": "语音合成失败"}
    else:
        base64_uri = "data:audio/mpeg; base64,"
        data = {"code": 200, "data": result, "base64_uri": base64_uri, "msg": "操作成功"}

    return JsonResponse(data)
```

## 4.11 接口方法路由

在django项目的urls.py中添加：

```python
# 导入接口模块
from api import upload_audio

# 添加接口路由
path('audio/upload', upload_audio.upload),
```

## 4.12 前端页面

前端页面可以通过AI工具生成，在deepseek中使用提示词完成前端页面生成

```
我要做一个前端页面，在页面中有如下功能：
1.能够采集用户的语音
2.将采集到的语音转换为base64，数据格式为mp3
```

```
3.调用django后台服务api接口
4.将django后台服务返回的base64编码转换为语音播放，格式为mp3
5.可以有重置功能
6.api接口定义如下：
url：http://localhost:8000/audio/upload
method：POST
params：{
    "base64_uri": "data:audio/mpeg; base64," ,
    "data":"base64编码内容"
}
response:{
    "code":"200",
    "msg":"操作成功",
    "data":"data:audio/mpeg; base64,XXXX"
}
帮我把页面实现，给出完整代码，注意不要擅自制造模拟数据
```

调试版本：

```html
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>语音采集与处理系统</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
            font-family: 'Segoe UI', 'Microsoft YaHei', sans-serif;
        }

        body {
            background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
            min-height: 100vh;
            padding: 20px;
            display: flex;
            justify-content: center;
            align-items: center;
        }

        .container {
            width: 100%;
            max-width: 800px;
            background-color: white;
            border-radius: 20px;
            box-shadow: 0 15px 35px rgba(0, 0, 0, 0.1);
            overflow: hidden;
        }

        .header {
            background: linear-gradient(90deg, #4b6cb7 0%, #182848 100%);
            color: white;
            padding: 25px 30px;
```

```
        text-align: center;
    }

    .header h1 {
        font-size: 28px;
        margin-bottom: 8px;
        font-weight: 600;
    }

    .header p {
        opacity: 0.9;
        font-size: 16px;
    }

    .content {
        padding: 30px;
    }

    .section {
        margin-bottom: 30px;
    }

    .section-title {
        font-size: 20px;
        color: #333;
        margin-bottom: 15px;
        padding-bottom: 8px;
        border-bottom: 2px solid #f0f0f0;
        display: flex;
        align-items: center;
    }

    .section-title i {
        margin-right: 10px;
        color: #4b6cb7;
    }

    .control-panel {
        display: flex;
        flex-wrap: wrap;
        gap: 15px;
        margin-bottom: 20px;
    }

    .btn {
        padding: 12px 24px;
        border: none;
        border-radius: 8px;
        font-size: 16px;
        font-weight: 600;
        cursor: pointer;
        transition: all 0.3s ease;
        display: flex;
        align-items: center;
        justify-content: center;
```

```
}

.btn i {
    margin-right: 8px;
}

.btn-primary {
    background-color: #4b6cb7;
    color: white;
}

.btn-primary:hover {
    background-color: #3a5795;
    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgba(74, 108, 183, 0.3);
}

.btn-secondary {
    background-color: #6c757d;
    color: white;
}

.btn-secondary:hover {
    background-color: #5a6268;
    transform: translateY(-2px);
}

.btn-success {
    background-color: #28a745;
    color: white;
}

.btn-success:hover {
    background-color: #218838;
    transform: translateY(-2px);
}

.btn-danger {
    background-color: #dc3545;
    color: white;
}

.btn-danger:hover {
    background-color: #c82333;
    transform: translateY(-2px);
}

.btn:disabled {
    background-color: #cccccc;
    cursor: not-allowed;
    transform: none !important;
    box-shadow: none !important;
}

.status-indicator {
```

```
        padding: 15px;
        border-radius: 8px;
        margin-bottom: 20px;
        display: flex;
        align-items: center;
        font-weight: 500;
    }

    .status-recording {
        background-color: rgba(220, 53, 69, 0.1);
        color: #dc3545;
        border-left: 4px solid #dc3545;
    }

    .status-processing {
        background-color: rgba(255, 193, 7, 0.1);
        color: #856404;
        border-left: 4px solid #ffc107;
    }

    .status-ready {
        background-color: rgba(40, 167, 69, 0.1);
        color: #155724;
        border-left: 4px solid #28a745;
    }

    .status-indicator i {
        margin-right: 10px;
        font-size: 18px;
    }

    .audio-visualizer {
        height: 100px;
        background-color: #f8f9fa;
        border-radius: 8px;
        margin-bottom: 20px;
        display: flex;
        align-items: center;
        justify-content: center;
        overflow: hidden;
        position: relative;
    }

    .visualizer-canvas {
        width: 100%;
        height: 100%;
    }

    .no-visualizer {
        color: #6c757d;
        font-style: italic;
    }

    .audio-player {
        width: 100%;
```

```
        margin-top: 10px;
        border-radius: 8px;
        background-color: #f8f9fa;
        padding: 10px;
    }

    .response-info {
        background-color: #f8f9fa;
        border-radius: 8px;
        padding: 15px;
        margin-top: 20px;
        font-family: monospace;
        font-size: 14px;
        max-height: 200px;
        overflow-y: auto;
    }

    .response-info h4 {
        margin-bottom: 10px;
        color: #333;
    }

    .response-data {
        color: #28a745;
    }

    .response-error {
        color: #dc3545;
    }

    .hidden {
        display: none;
    }

    .footer {
        text-align: center;
        padding: 20px;
        color: #6c757d;
        font-size: 14px;
        border-top: 1px solid #f0f0f0;
    }

    @media (max-width: 768px) {
        .container {
            border-radius: 10px;
        }

        .header {
            padding: 20px;
        }

        .content {
            padding: 20px;
        }
```

```
            .control-panel {
                flex-direction: column;
            }

            .btn {
                width: 100%;
            }
        }
    </style>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css">
</head>
<body>
    <div class="container">
        <div class="header">
            <h1><i class="fas fa-microphone-alt"></i> 语音采集与处理系统</h1>
            <p>采集语音 → 转换为Base64 → 发送到Django API → 播放返回的语音</p>
        </div>

        <div class="content">
            <!-- 状态指示器 -->
            <div id="statusIndicator" class="status-indicator status-ready">
                <i class="fas fa-check-circle"></i>
                <span>准备就绪，点击"开始录音"按钮开始采集语音</span>
            </div>

            <!-- 语音可视化 -->
            <div class="audio-visualizer">
                <canvas id="visualizer" class="visualizer-canvas"></canvas>
                <div id="noVisualizer" class="no-visualizer">
                    录音时可视化将显示在这里
                </div>
            </div>

            <!-- 控制面板 -->
            <div class="section">
                <h3 class="section-title"><i class="fas fa-sliders-h"></i> 控制面
板</h3>
                <div class="control-panel">
                    <button id="startBtn" class="btn btn-primary">
                        <i class="fas fa-microphone"></i> 开始录音
                    </button>
                    <button id="stopBtn" class="btn btn-danger" disabled>
                        <i class="fas fa-stop"></i> 停止录音
                    </button>
                    <button id="sendBtn" class="btn btn-success" disabled>
                        <i class="fas fa-paper-plane"></i> 发送到API
                    </button>
                    <button id="playBtn" class="btn btn-secondary" disabled>
                        <i class="fas fa-play"></i> 播放返回的语音
                    </button>
                    <button id="resetBtn" class="btn btn-secondary">
                        <i class="fas fa-redo"></i> 重置
                    </button>
                </div>
```

```
            </div>

            <!-- 音频播放器 -->
            <div class="section">
                <h3 class="section-title"><i class="fas fa-volume-up"></i> 语音播
放</h3>
                <audio id="audioPlayer" class="audio-player" controls></audio>
                <div id="noAudio" class="no-visualizer">
                    返回的语音将在这里播放
                </div>
            </div>

            <!-- API响应信息 -->
            <div class="section">
                <h3 class="section-title"><i class="fas fa-code"></i> API响应
</h3>
                <div id="responseInfo" class="response-info">
                    <h4>API响应将显示在这里：</h4>
                    <div id="responseData">尚未发送请求</div>
                </div>
            </div>
        </div>

        <div class="footer">
            <p>语音采集与处理系统 &copy; 2023 | 支持Chrome、Edge等现代浏览器</p>
        </div>
    </div>

    <script>
        // DOM元素
        const startBtn = document.getElementById('startBtn');
        const stopBtn = document.getElementById('stopBtn');
        const sendBtn = document.getElementById('sendBtn');
        const playBtn = document.getElementById('playBtn');
        const resetBtn = document.getElementById('resetBtn');
        const statusIndicator = document.getElementById('statusIndicator');
        const audioPlayer = document.getElementById('audioPlayer');
        const noAudio = document.getElementById('noAudio');
        const visualizerCanvas = document.getElementById('visualizer');
        const noVisualizer = document.getElementById('noVisualizer');
        const responseData = document.getElementById('responseData');

        // 全局变量
        let mediaRecorder;
        let audioChunks = [];
        let audioBlob;
        let audioBase64;
        let apiResponseAudio;
        let audioContext;
        let analyser;
        let dataArray;
        let canvasCtx;
        let isRecording = false;
        let animationId;
```

```
        // 初始化Canvas上下文
        if (visualizerCanvas) {
            canvasCtx = visualizerCanvas.getContext('2d');
        }

        // 更新状态指示器
        function updateStatus(status, message) {
            statusIndicator.className = 'status-indicator';

            if (status === 'recording') {
                statusIndicator.classList.add('status-recording');
                statusIndicator.innerHTML = `<i class="fas fa-microphone"></i>
<span>${message}</span>`;
            } else if (status === 'processing') {
                statusIndicator.classList.add('status-processing');
                statusIndicator.innerHTML = `<i class="fas fa-cog fa-spin"></i>
<span>${message}</span>`;
            } else {
                statusIndicator.classList.add('status-ready');
                statusIndicator.innerHTML = `<i class="fas fa-check-circle"></i>
<span>${message}</span>`;
            }
        }

        // 开始录音
        async function startRecording() {
            try {
                // 请求麦克风权限
                const stream = await navigator.mediaDevices.getUserMedia({
audio: true });

                // 创建媒体录制器
                mediaRecorder = new MediaRecorder(stream);

                // 初始化音频上下文和可视化
                if (visualizerCanvas) {
                    initAudioVisualizer(stream);
                }

                // 收集音频数据
                mediaRecorder.ondataavailable = (event) => {
                    audioChunks.push(event.data);
                };

                // 录音完成
                mediaRecorder.onstop = () => {
                    audioBlob = new Blob(audioChunks, { type: 'audio/mp3' });

                    // 将Blob转换为Base64
                    const reader = new FileReader();
                    reader.readAsDataURL(audioBlob);
                    reader.onloadend = () => {
                        audioBase64 = reader.result;

                        // 更新UI
```

```
                        sendBtn.disabled = false;
                        updateStatus('ready', `录音完成，时长:
${Math.floor(audioBlob.size / 1000)} KB`);

                        // 停止可视化
                        if (animationId) {
                            cancelAnimationFrame(animationId);
                        }

                        if (audioContext && audioContext.state !== 'closed') {
                            audioContext.close();
                        }
                    };
                };

                // 开始录制
                mediaRecorder.start();
                isRecording = true;

                // 更新UI
                startBtn.disabled = true;
                stopBtn.disabled = false;
                sendBtn.disabled = true;
                playBtn.disabled = true;
                updateStatus('recording', '正在录音... 点击"停止录音"按钮结束');

            } catch (error) {
                console.error('录音失败:', error);
                updateStatus('ready', `录音失败: ${error.message}`);
                alert('无法访问麦克风，请确保已授予麦克风权限并刷新页面重试。');
            }
        }

        // 停止录音
        function stopRecording() {
            if (mediaRecorder && isRecording) {
                mediaRecorder.stop();
                isRecording = false;

                // 停止所有音频轨道
                mediaRecorder.stream.getTracks().forEach(track => track.stop());

                // 更新UI
                startBtn.disabled = false;
                stopBtn.disabled = true;
            }
        }

        // 初始化音频可视化
        function initAudioVisualizer(stream) {
            audioContext = new (window.AudioContext ||
window.webkitAudioContext)();
            analyser = audioContext.createAnalyser();
            const source = audioContext.createMediaStreamSource(stream);
            source.connect(analyser);
```

```
            analyser.fftSize = 256;
            const bufferLength = analyser.frequencyBinCount;
            dataArray = new Uint8Array(bufferLength);

            // 显示Canvas，隐藏提示
            visualizerCanvas.classList.remove('hidden');
            noVisualizer.classList.add('hidden');

            // 开始绘制可视化
            drawVisualizer();
        }

        // 绘制音频可视化
        function drawVisualizer() {
            if (!analyser || !canvasCtx) return;

            animationId = requestAnimationFrame(drawVisualizer);
            analyser.getByteFrequencyData(dataArray);

            canvasCtx.fillStyle = 'rgb(248, 249, 250)';
            canvasCtx.fillRect(0, 0, visualizerCanvas.width,
visualizerCanvas.height);

            const barWidth = (visualizerCanvas.width / dataArray.length) * 2.5;
            let barHeight;
            let x = 0;

            for (let i = 0; i < dataArray.length; i++) {
                barHeight = dataArray[i] / 2;

                // 使用渐变色
                const gradient = canvasCtx.createLinearGradient(0, 0, 0,
visualizerCanvas.height);
                gradient.addColorStop(0, '#4b6cb7');
                gradient.addColorStop(1, '#182848');

                canvasCtx.fillStyle = gradient;
                canvasCtx.fillRect(x, visualizerCanvas.height - barHeight,
barWidth, barHeight);

                x += barWidth + 1;
            }
        }

        // 发送数据到Django API
        async function sendToAPI() {
            if (!audioBase64) {
                alert('请先录制语音');
                return;
            }

            // 更新状态
            updateStatus('processing', '正在发送数据到服务器...');
            sendBtn.disabled = true;
```

```
            // 准备请求数据
            const base64Data = audioBase64.split(',')[1]; // 移除
data:audio/mp3; base64,前缀
            const requestData = {
                base64_uri: "data:audio/mpeg; base64,",
                data: base64Data
            };

            try {
                // 发送请求到Django API
                const response = await
fetch('http://localhost:8000/audio/upload', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json',
                    },
                    body: JSON.stringify(requestData)
                });

                const result = await response.json();

                // 显示响应
                if (result.code == "200") {
                    responseData.innerHTML = `<span class="response-
data">${result.msg}</span>`;
                    responseData.innerHTML += `<br><br><strong>返回的Base64数据前
100字符：</strong><br>${result.data.substring(0, 100)}...`;

                    // 保存返回的音频数据
                    apiResponseAudio = result.base64_uri + result.data;

                    // 启用播放按钮
                    playBtn.disabled = false;
                    updateStatus('ready', '服务器处理成功，可以播放返回的语音');
                } else {
                    responseData.innerHTML = `<span class="response-error">错误:
${result.msg || '未知错误'}</span>`;
                    updateStatus('ready', '服务器处理失败');
                }

            } catch (error) {
                console.error('API请求失败:', error);
                responseData.innerHTML = `<span class="response-error">API请求失
败: ${error.message}</span>`;
                updateStatus('ready', 'API请求失败，请检查网络连接和Django服务');
            } finally {
                sendBtn.disabled = false;
            }
        }

        // 播放返回的语音
        function playResponseAudio() {
            if (!apiResponseAudio) {
                alert('没有可播放的音频数据');
```

```
                return;
            }

            // 设置音频源
            audioPlayer.src = apiResponseAudio;

            // 显示音频播放器，隐藏提示
            audioPlayer.classList.remove('hidden');
            noAudio.classList.add('hidden');

            // 播放音频
            audioPlayer.play().catch(error => {
                console.error('播放失败:', error);
                alert('音频播放失败，请检查音频数据格式');
            });
        }

        // 重置所有状态
        function resetAll() {
            // 停止录音
            if (isRecording && mediaRecorder) {
                mediaRecorder.stop();
                isRecording = false;
            }

            // 停止音频播放
            audioPlayer.pause();
            audioPlayer.src = '';
            audioPlayer.classList.add('hidden');
            noAudio.classList.remove('hidden');

            // 清除Canvas
            if (canvasCtx) {
                canvasCtx.clearRect(0, 0, visualizerCanvas.width,
visualizerCanvas.height);
            }

            // 隐藏Canvas，显示提示
            visualizerCanvas.classList.add('hidden');
            noVisualizer.classList.remove('hidden');

            // 停止可视化动画
            if (animationId) {
                cancelAnimationFrame(animationId);
            }

            // 关闭音频上下文
            if (audioContext && audioContext.state !== 'closed') {
                audioContext.close();
            }

            // 重置变量
            mediaRecorder = null;
            audioChunks = [];
            audioBlob = null;
```

```
            audioBase64 = null;
            apiResponseAudio = null;
            analyser = null;
            dataArray = null;

            // 更新UI
            startBtn.disabled = false;
            stopBtn.disabled = true;
            sendBtn.disabled = true;
            playBtn.disabled = true;

            // 清除响应数据
            responseData.textContent = '尚未发送请求';

            updateStatus('ready', '准备就绪，点击"开始录音"按钮开始采集语音');
        }

        // 事件监听器
        startBtn.addEventListener('click', startRecording);
        stopBtn.addEventListener('click', stopRecording);
        sendBtn.addEventListener('click', sendToAPI);
        playBtn.addEventListener('click', playResponseAudio);
        resetBtn.addEventListener('click', resetAll);

        // 页面加载完成时初始化
        window.addEventListener('load', () => {
            updateStatus('ready', '准备就绪，点击"开始录音"按钮开始采集语音');

            // 检查浏览器是否支持MediaRecorder
            if (!navigator.mediaDevices || !navigator.mediaDevices.getUserMedia)
{
                alert('您的浏览器不支持音频录制功能，请使用Chrome、Edge等现代浏览器。');
                startBtn.disabled = true;
            }
        });
    </script>
</body>
</html>
```

简易版本:

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>语音采集与处理系统</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
            font-family: 'Segoe UI', 'Microsoft YaHei', sans-serif;
        }
```

```
        body {
            background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
            min-height: 100vh;
            padding: 20px;
            display: flex;
            justify-content: center;
            align-items: center;
        }

        .container {
            width: 100%;
            max-width: 800px;
            background-color: white;
            border-radius: 20px;
            box-shadow: 0 15px 35px rgba(0, 0, 0, 0.1);
            overflow: hidden;
        }

        .header {
            background: linear-gradient(90deg, #4b6cb7 0%, #182848 100%);
            color: white;
            padding: 25px 30px;
            text-align: center;
        }

        .header h1 {
            font-size: 28px;
            margin-bottom: 8px;
            font-weight: 600;
        }

        .header p {
            opacity: 0.9;
            font-size: 16px;
        }

        .content {
            padding: 30px;
        }

        .section {
            margin-bottom: 30px;
        }

        .section-title {
            font-size: 20px;
            color: #333;
            margin-bottom: 15px;
            padding-bottom: 8px;
            border-bottom: 2px solid #f0f0f0;
            display: flex;
            align-items: center;
        }

        .section-title i {
```

```
        margin-right: 10px;
        color: #4b6cb7;
    }

    .control-panel {
        display: flex;
        justify-content: center;
        gap: 20px;
        margin-bottom: 20px;
    }

    .btn {
        padding: 15px 30px;
        border: none;
        border-radius: 8px;
        font-size: 18px;
        font-weight: 600;
        cursor: pointer;
        transition: all 0.3s ease;
        display: flex;
        align-items: center;
        justify-content: center;
        min-width: 150px;
    }

    .btn i {
        margin-right: 10px;
    }

    .btn-primary {
        background-color: #4b6cb7;
        color: white;
    }

    .btn-primary:hover {
        background-color: #3a5795;
        transform: translateY(-2px);
        box-shadow: 0 5px 15px rgba(74, 108, 183, 0.3);
    }

    .btn-danger {
        background-color: #dc3545;
        color: white;
    }

    .btn-danger:hover {
        background-color: #c82333;
        transform: translateY(-2px);
    }

    .btn:disabled {
        background-color: #cccccc;
        cursor: not-allowed;
        transform: none !important;
        box-shadow: none !important;
```

```
}

.status-indicator {
    padding: 15px;
    border-radius: 8px;
    margin-bottom: 20px;
    display: flex;
    align-items: center;
    font-weight: 500;
}

.status-recording {
    background-color: rgba(220, 53, 69, 0.1);
    color: #dc3545;
    border-left: 4px solid #dc3545;
}

.status-processing {
    background-color: rgba(255, 193, 7, 0.1);
    color: #856404;
    border-left: 4px solid #ffc107;
}

.status-ready {
    background-color: rgba(40, 167, 69, 0.1);
    color: #155724;
    border-left: 4px solid #28a745;
}

.status-indicator i {
    margin-right: 10px;
    font-size: 18px;
}

.audio-visualizer {
    height: 100px;
    background-color: #f8f9fa;
    border-radius: 8px;
    margin-bottom: 20px;
    display: flex;
    align-items: center;
    justify-content: center;
    overflow: hidden;
    position: relative;
}

.visualizer-canvas {
    width: 100%;
    height: 100%;
}

.no-visualizer {
    color: #6c757d;
    font-style: italic;
}
```

```
        .audio-player {
            width: 100%;
            margin-top: 10px;
            border-radius: 8px;
            background-color: #f8f9fa;
            padding: 10px;
        }

        .hidden {
            display: none;
        }

        .footer {
            text-align: center;
            padding: 20px;
            color: #6c757d;
            font-size: 14px;
            border-top: 1px solid #f0f0f0;
        }

        @media (max-width: 768px) {
            .container {
                border-radius: 10px;
            }

            .header {
                padding: 20px;
            }

            .content {
                padding: 20px;
            }

            .control-panel {
                flex-direction: column;
                align-items: center;
            }

            .btn {
                width: 100%;
                max-width: 250px;
            }
        }
    </style>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css">
</head>
<body>
    <div class="container">
        <div class="header">
            <h1><i class="fas fa-microphone-alt"></i> 语音采集与处理系统</h1>
            <p>点击开始录音 → 停止后自动发送到后台 → 播放返回的语音</p>
        </div>
```

```
        <div class="content">
            <!-- 状态指示器 -->
            <div id="statusIndicator" class="status-indicator status-ready">
                <i class="fas fa-check-circle"></i>
                <span>准备就绪，点击"开始录音"按钮</span>
            </div>

            <!-- 语音可视化 -->
            <div class="audio-visualizer">
                <canvas id="visualizer" class="visualizer-canvas hidden">
</canvas>
                <div id="noVisualizer" class="no-visualizer">
                    开始录音后将显示音频可视化
                </div>
            </div>

            <!-- 控制面板 -->
            <div class="section">
                <h3 class="section-title"><i class="fas fa-sliders-h"></i> 控制面
板</h3>
                <div class="control-panel">
                    <button id="startBtn" class="btn btn-primary">
                        <i class="fas fa-microphone"></i> 开始录音
                    </button>
                    <button id="stopBtn" class="btn btn-danger" disabled>
                        <i class="fas fa-stop"></i> 停止录音
                    </button>
                </div>
            </div>

            <!-- 音频播放器 -->
            <div class="section">
                <h3 class="section-title"><i class="fas fa-volume-up"></i> 返回的
语音</h3>
                <audio id="audioPlayer" class="audio-player hidden" controls>
</audio>
                <div id="noAudio" class="no-visualizer">
                    停止录音后，返回的语音将自动播放
                </div>
            </div>
        </div>

        <div class="footer">
            <p>语音采集与处理系统 &copy; 2023 | 支持Chrome、Edge等现代浏览器</p>
        </div>
    </div>

    <script>
        // DOM元素
        const startBtn = document.getElementById('startBtn');
        const stopBtn = document.getElementById('stopBtn');
        const statusIndicator = document.getElementById('statusIndicator');
        const audioPlayer = document.getElementById('audioPlayer');
        const noAudio = document.getElementById('noAudio');
        const visualizerCanvas = document.getElementById('visualizer');
```

```
        const noVisualizer = document.getElementById('noVisualizer');

        // 全局变量
        let mediaRecorder;
        let audioChunks = [];
        let audioBlob;
        let audioBase64;
        let audioContext;
        let analyser;
        let dataArray;
        let canvasCtx;
        let isRecording = false;
        let animationId;

        // 初始化Canvas上下文
        if (visualizerCanvas) {
            canvasCtx = visualizerCanvas.getContext('2d');
        }

        // 更新状态指示器
        function updateStatus(status, message) {
            statusIndicator.className = 'status-indicator';

            if (status === 'recording') {
                statusIndicator.classList.add('status-recording');
                statusIndicator.innerHTML = `<i class="fas fa-microphone"></i>
<span>${message}</span>`;
            } else if (status === 'processing') {
                statusIndicator.classList.add('status-processing');
                statusIndicator.innerHTML = `<i class="fas fa-cog fa-spin"></i>
<span>${message}</span>`;
            } else {
                statusIndicator.classList.add('status-ready');
                statusIndicator.innerHTML = `<i class="fas fa-check-circle"></i>
<span>${message}</span>`;
            }
        }

        // 开始录音
        async function startRecording() {
            try {
                // 首先清除之前的录音数据
                clearRecordingData();

                // 请求麦克风权限
                const stream = await navigator.mediaDevices.getUserMedia({
audio: true });

                // 创建媒体录制器
                mediaRecorder = new MediaRecorder(stream);

                // 初始化音频上下文和可视化
                if (visualizerCanvas) {
                    initAudioVisualizer(stream);
                }
```

```
        // 收集音频数据
        mediaRecorder.ondataavailable = (event) => {
            audioChunks.push(event.data);
        };

        // 录音完成
        mediaRecorder.onstop = async () => {
            audioBlob = new Blob(audioChunks, { type: 'audio/mp3' });

            // 将Blob转换为Base64
            const reader = new FileReader();
            reader.readAsDataURL(audioBlob);
            reader.onloadend = async () => {
                audioBase64 = reader.result;

                // 更新UI
                updateStatus('processing', '录音完成，正在发送到服务器...');

                // 停止可视化
                if (animationId) {
                    cancelAnimationFrame(animationId);
                }

                if (audioContext && audioContext.state !== 'closed') {
                    audioContext.close();
                }

                // 自动发送到API
                await sendToAPI();
            };
        };

        // 开始录制
        mediaRecorder.start();
        isRecording = true;

        // 更新UI
        startBtn.disabled = true;
        stopBtn.disabled = false;
        updateStatus('recording', '正在录音... 点击"停止录音"按钮结束');

    } catch (error) {
        console.error('录音失败:', error);
        updateStatus('ready', `录音失败: ${error.message}`);
        alert('无法访问麦克风，请确保已授予麦克风权限并刷新页面重试。');
    }
}

// 停止录音
function stopRecording() {
    if (mediaRecorder && isRecording) {
        mediaRecorder.stop();
        isRecording = false;
```

```
                // 停止所有音频轨道
                mediaRecorder.stream.getTracks().forEach(track => track.stop());

                // 更新UI
                startBtn.disabled = false;
                stopBtn.disabled = true;
            }
        }

        // 清除录音数据
        function clearRecordingData() {
            // 停止可视化动画
            if (animationId) {
                cancelAnimationFrame(animationId);
            }

            // 关闭音频上下文
            if (audioContext && audioContext.state !== 'closed') {
                audioContext.close();
            }

            // 清除Canvas
            if (canvasCtx && visualizerCanvas) {
                canvasCtx.clearRect(0, 0, visualizerCanvas.width,
visualizerCanvas.height);
            }

            // 隐藏Canvas，显示提示
            visualizerCanvas.classList.add('hidden');
            noVisualizer.classList.remove('hidden');

            // 重置变量
            mediaRecorder = null;
            audioChunks = [];
            audioBlob = null;
            audioBase64 = null;
            analyser = null;
            dataArray = null;
            isRecording = false;
        }

        // 初始化音频可视化
        function initAudioVisualizer(stream) {
            audioContext = new (window.AudioContext ||
window.webkitAudioContext)();
            analyser = audioContext.createAnalyser();
            const source = audioContext.createMediaStreamSource(stream);
            source.connect(analyser);

            analyser.fftSize = 256;
            const bufferLength = analyser.frequencyBinCount;
            dataArray = new Uint8Array(bufferLength);

            // 显示Canvas，隐藏提示
            visualizerCanvas.classList.remove('hidden');
```

```
            noVisualizer.classList.add('hidden');

            // 调整Canvas大小
            visualizerCanvas.width = visualizerCanvas.offsetWidth;
            visualizerCanvas.height = visualizerCanvas.offsetHeight;

            // 开始绘制可视化
            drawVisualizer();
        }

        // 绘制音频可视化
        function drawVisualizer() {
            if (!analyser || !canvasCtx) return;

            animationId = requestAnimationFrame(drawVisualizer);
            analyser.getByteFrequencyData(dataArray);

            canvasCtx.fillStyle = 'rgb(248, 249, 250)';
            canvasCtx.fillRect(0, 0, visualizerCanvas.width,
visualizerCanvas.height);

            const barWidth = (visualizerCanvas.width / dataArray.length) * 2.5;
            let barHeight;
            let x = 0;

            for (let i = 0; i < dataArray.length; i++) {
                barHeight = dataArray[i] / 2;

                // 使用渐变色
                const gradient = canvasCtx.createLinearGradient(0, 0, 0,
visualizerCanvas.height);
                gradient.addColorStop(0, '#4b6cb7');
                gradient.addColorStop(1, '#182848');

                canvasCtx.fillStyle = gradient;
                canvasCtx.fillRect(x, visualizerCanvas.height - barHeight,
barWidth, barHeight);

                x += barWidth + 1;
            }
        }

        // 发送数据到Django API（保持原有逻辑不变）
        async function sendToAPI() {
            if (!audioBase64) {
                alert('请先录制语音');
                return;
            }

            // 准备请求数据
            const base64Data = audioBase64.split(',')[1]; // 移除
data:audio/mp3; base64,前缀
            const requestData = {
                base64_uri: "data:audio/mpeg; base64,",
                data: base64Data
```

```
            };

            try {
                // 发送请求到Django API
                const response = await
fetch('http://localhost:8000/audio/upload', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json',
                    },
                    body: JSON.stringify(requestData)
                });

                const result = await response.json();

                // 处理响应
                if (result.code == "200") {
                    // 保存返回的音频数据
                    const apiResponseAudio = result.base64_uri + result.data;

                    // 自动播放返回的音频
                    playResponseAudio(apiResponseAudio);
                    updateStatus('ready', '服务器处理成功，正在播放返回的语音');
                } else {
                    updateStatus('ready', '服务器处理失败');
                }

                // 每次API请求完成后清除录音数据
                clearRecordingData();

            } catch (error) {
                console.error('API请求失败:', error);
                updateStatus('ready', 'API请求失败，请检查网络连接和Django服务');

                // API请求失败时也清除录音数据
                clearRecordingData();
            }
        }

        // 播放返回的语音
        function playResponseAudio(audioData) {
            if (!audioData) {
                alert('没有可播放的音频数据');
                return;
            }

            // 设置音频源
            audioPlayer.src = audioData;

            // 显示音频播放器，隐藏提示
            audioPlayer.classList.remove('hidden');
            noAudio.classList.add('hidden');

            // 播放音频
            audioPlayer.play().catch(error => {
```

```
                console.error('播放失败:', error);
                alert('音频播放失败，请检查音频数据格式');
            });
        }

        // 事件监听器
        startBtn.addEventListener('click', startRecording);
        stopBtn.addEventListener('click', stopRecording);

        // 页面加载完成时初始化
        window.addEventListener('load', () => {
            updateStatus('ready', '准备就绪，点击"开始录音"按钮');

            // 检查浏览器是否支持MediaRecorder
            if (!navigator.mediaDevices || !navigator.mediaDevices.getUserMedia)
{
                alert('您的浏览器不支持音频录制功能，请使用Chrome、Edge等现代浏览器。');
                startBtn.disabled = true;
            }

            // 监听窗口大小变化，调整Canvas
            window.addEventListener('resize', () => {
                if (visualizerCanvas &&
!visualizerCanvas.classList.contains('hidden')) {
                    visualizerCanvas.width = visualizerCanvas.offsetWidth;
                    visualizerCanvas.height = visualizerCanvas.offsetHeight;
                }
            });
        });
    </script>
</body>
</html>
```